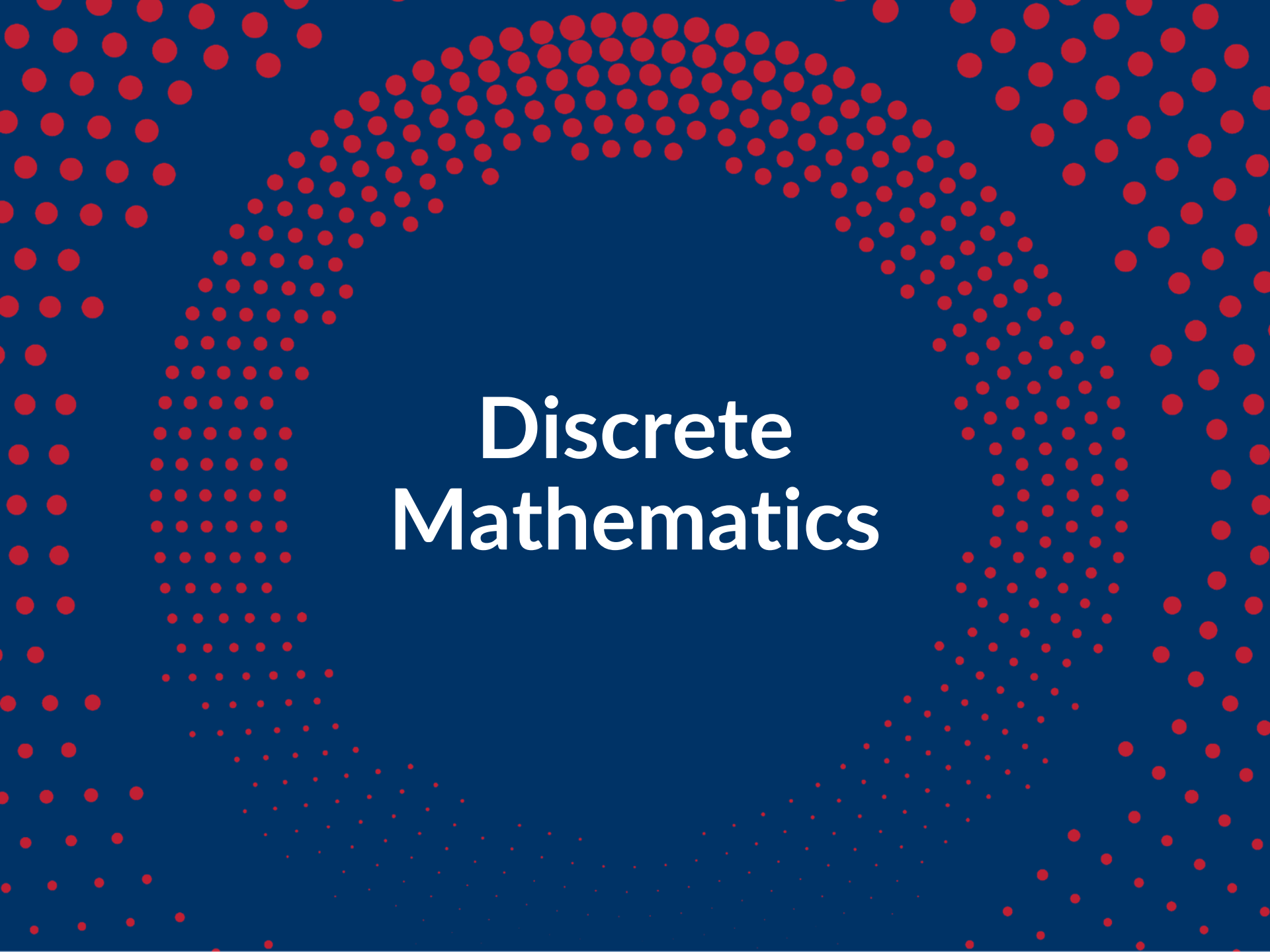


The background of the entire image is a dark blue field filled with a pattern of red dots. These dots are arranged in a way that they form a large, faint, stylized 'H' shape that frames the central text. The dots are of varying sizes and are more densely packed in some areas, creating a sense of depth and movement.

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



Discrete Mathematics

PART 1

COMBINATORIAL THEORY

(Lý thuyết tổ hợp)

PART 2

GRAPH THEORY

(Lý thuyết đồ thị)

CONTENT

- Chapter 1. Fundamental concepts
- Chapter 2. Graph representation
- Chapter 3. Graph Traversal
- Chapter 4. Tree and Spanning tree
- **Chapter 5. Shortest path problem**
- Chapter 6. Maximum flow problem



1. Shortest path problem
2. Shortest path properties, Reduce upper bound
3. Bellman-Ford algorithm
4. Dijkstra algorithm
5. Shortest path in acyclic graph
6. Floyd-Warshall algorithm

1. Shortest Path

- Generalize distance to weighted setting
- Digraph $G = (V, E)$ with weight function $W: E \rightarrow R$ (assigning real values to edges)

$$w(v_1, v_2) \quad w(v_2, v_3) \quad w(v_{k-1}, v_k)$$

- Weight of path $p = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ is

$$w(p) = \sum_{i=1}^{k-1} w(v_i, v_{i+1})$$

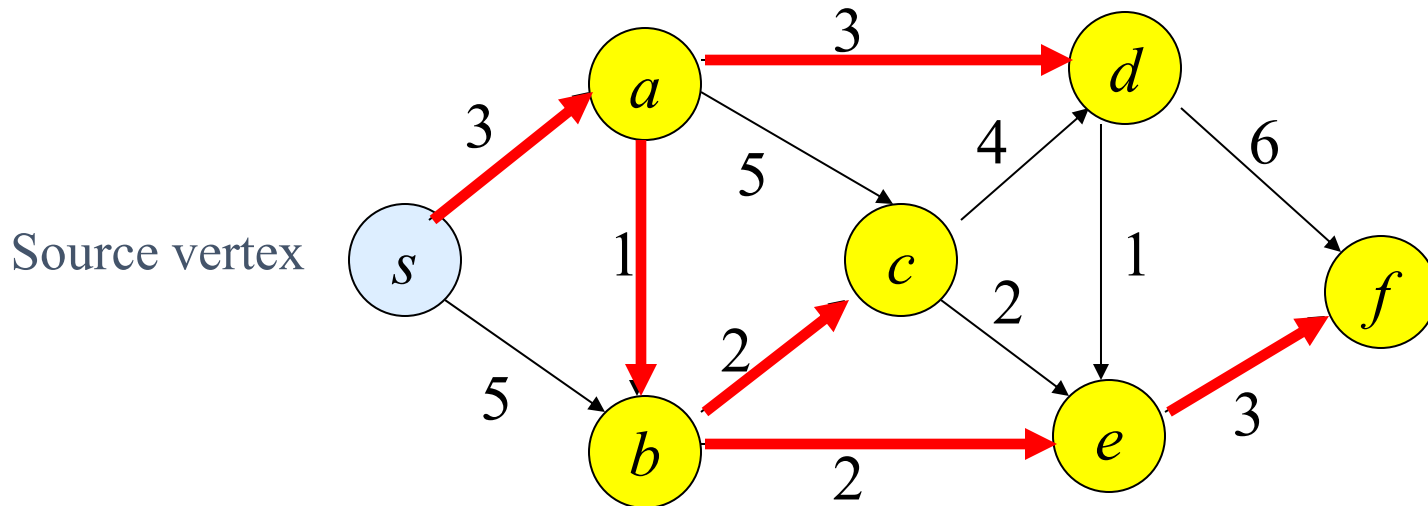
- Shortest path = a path of the minimum weight

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \rightsquigarrow^p v\}; & \text{if there is a path from } u \text{ to } v, \\ \infty & \text{otherwise.} \end{cases}$$

- Applications
 - static/dynamic network routing
 - robot motion planning
 - map/route generation in traffic
 - speech interpretation (best interpretation of a spoken sentence)
 - medical imaging

Example

Graph $G = (V, E)$, source vertex $s \in V$, find the shortest path from s to each of remaining vertices.



	<i>s</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
path	<i>s</i>	<i>s,a</i>	<i>s,a,b</i>	<i>s,a,b,c</i>	<i>s,a,d</i>	<i>s,a,b,e</i>	<i>s,a,b,e,f</i>
weight	0	3	4	6	6	6	9

Shortest-Path Variants

- **Single-source shortest-paths problem**
 - Find a shortest path from a given source (vertex s) to each of the vertices.
- **Single-destination shortest-paths problem**
 - Find a shortest path to a given *destination* vertex t from each vertex v .
- **Single-pair shortest-path problem**
 - Given two vertices, find a shortest path between them. Solution to single-source problem solves this problem efficiently, too.
- **All-pairs shortest-paths problem**
 - Find a shortest path from u to v for every pair of vertices u and v

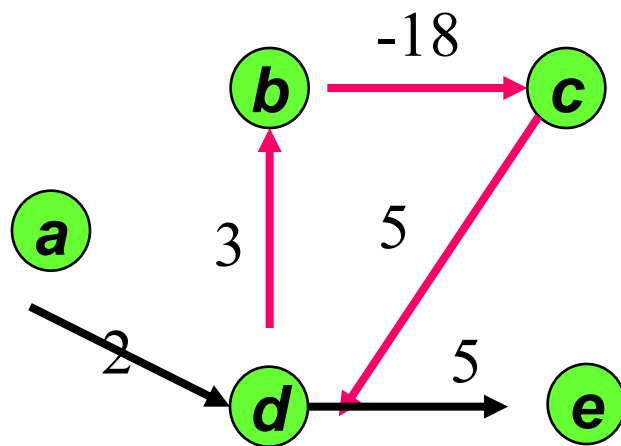
Comment:

- The problems are arranged in order from simple to complex
- Whenever there is an efficient algorithm for solving one of the three problems, the algorithm can also be used to solve the remaining two problems.



Negative Weights and Cycles?

- Negative edges are OK.
- Negative weight cycles: NO (otherwise paths with arbitrary small “lengths” would be possible)



Cycle: $(d \rightarrow b \rightarrow c \rightarrow d)$

Length = -10

Path from a to e :

$P: a \rightarrow \sigma(d \rightarrow b \rightarrow c \rightarrow d) \rightarrow e$

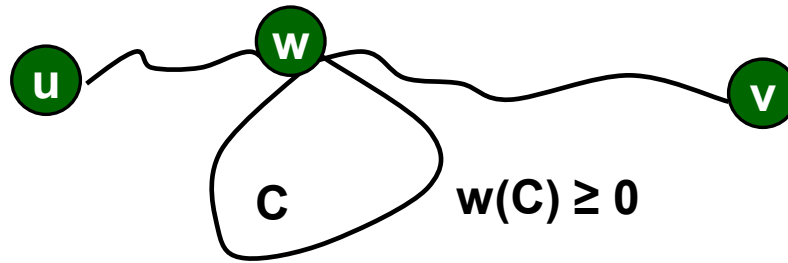
$w(P) = 7 - 10\sigma \rightarrow -\infty$, khi $\sigma \rightarrow +\infty$

Assumption:

Graph does not contain negative weight cycles

Properties of shortest paths

- **Property 1.** Shortest-paths can have no cycles (The shortest path can always be found among single paths Path where vertices are distinct.
Proof: Removing a cycle with positive length could reduce the length of the path.



- **Property 2.** Any shortest-path in graph G can not traverse through more than $n - 1$ edges, where n is the number of vertices

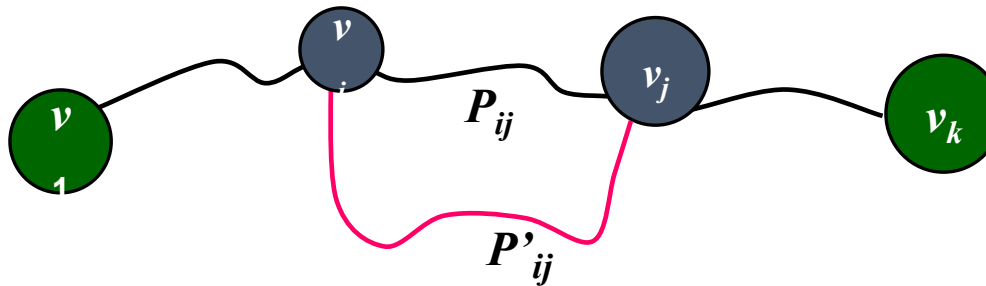
Properties of shortest paths

Property 3: Assume $P = \langle v_1, v_2, \dots, v_k \rangle$ is the shortest path from v_1 to v_k . Then, $P_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ is the shortest path from v_i to v_j , where $1 \leq i \leq j \leq k$.

(In words: **subpaths of shortest paths are also shortest paths**)

Proof (by contradiction). If P_{ij} is not the shortest path from v_i to v_j , then one can find P'_{ij} is the shortest path from v_i to v_j satisfying $w(P'_{ij}) < w(P_{ij})$. Then we get P' is the path obtained from P by substituting P_{ij} by P'_{ij} , thus:

$$w(P') < w(P) ?!$$



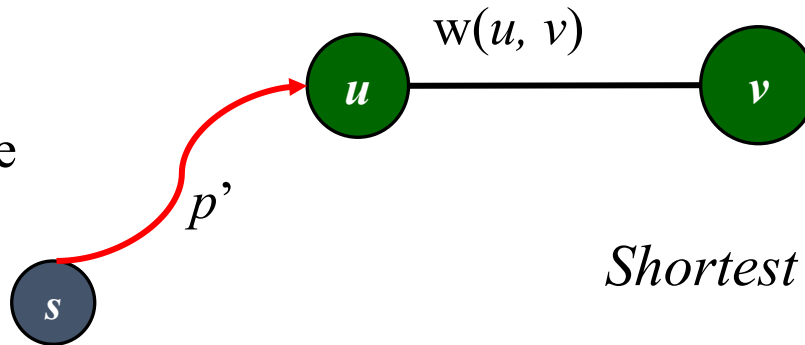
if some subpaths were not the shortest paths, one could substitute the shorter subpath and create a shorter total path

Properties of shortest paths

Denote: $\delta(u, v)$ = length of the shortest path from u to v

Assume P is the shortest path from s to v , where $P = s \xrightarrow{p'} \dots u \rightarrow v$.
Then $\delta(s, v) = \delta(s, u) + w(u, v)$.

$\delta(s, u)$: length of the
shortest path
from s to u



Shortest path from s to v :

$$P = s \xrightarrow{p'} \dots u \rightarrow v$$

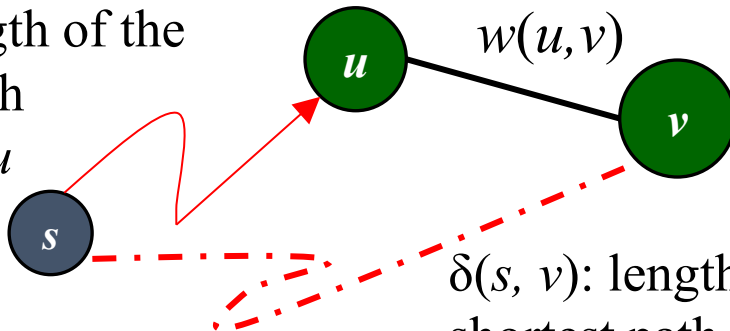
Properties of shortest paths

Denote: $\delta(u, v)$ = length of the shortest path from u to v

Assume P is the shortest path from s to v , where $P = s \dots \xrightarrow{p'} \dots u \rightarrow v$.
Then $\delta(s, v) = \delta(s, u) + w(u, v)$.

Property 4: Assume $s \in V$. For each edge $(u, v) \in E$, we have
 $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

$\delta(s, u)$: length of the
shortest path
from s to u



$\delta(s, v)$: length of the
shortest path from s to v

Shortest-Path Variants

- **Single-source shortest-paths problem**

- Find a shortest path from a given source (vertex s) to each of the vertices.

- **Single-destination shortest-paths problem**

- Find a shortest path to a given *destination* vertex t from each vertex v .

- **Single-pair shortest-path problem**

- Given two vertices, find a shortest path between them. Solution to single-source problem solves this problem efficiently, too.

- **All-pairs shortest-paths problem**

- Find a shortest path from u to v for every pair of vertices u and v

Comment:

- The problems are arranged in order from simple to complex
- Whenever there is an efficient algorithm for solving one of the three problems, the algorithm can also be used to solve the remaining two problems.



Shortest path representation

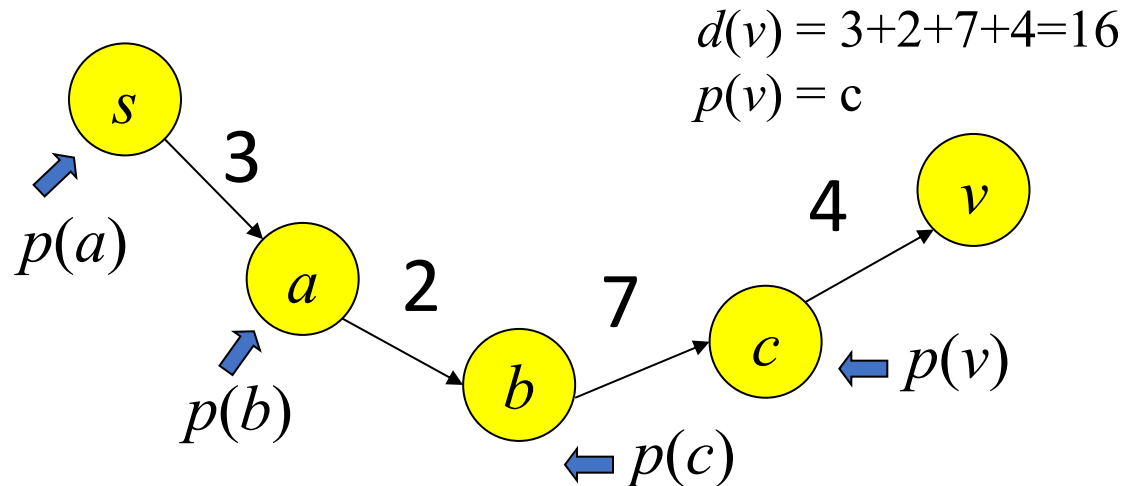
Shortest path algorithms works on 2 arrays:

- ✦ $d(v)$ = the length of shortest path from s to v that algorithm found so far
(upper bound for the length of the shortest path from s to v).
- ✦ $p(v)$ = a predecessor of v in this shortest path
(used to back trace the path from s to v).

$$\delta(s, v) \leq d(v)$$

Initialization

```
for  $v \in V(G)$   
  do  $d[v] \leftarrow \infty$   
      $p[v] \leftarrow \text{NULL}$   
 $d[s] \leftarrow 0$ 
```



Relaxation

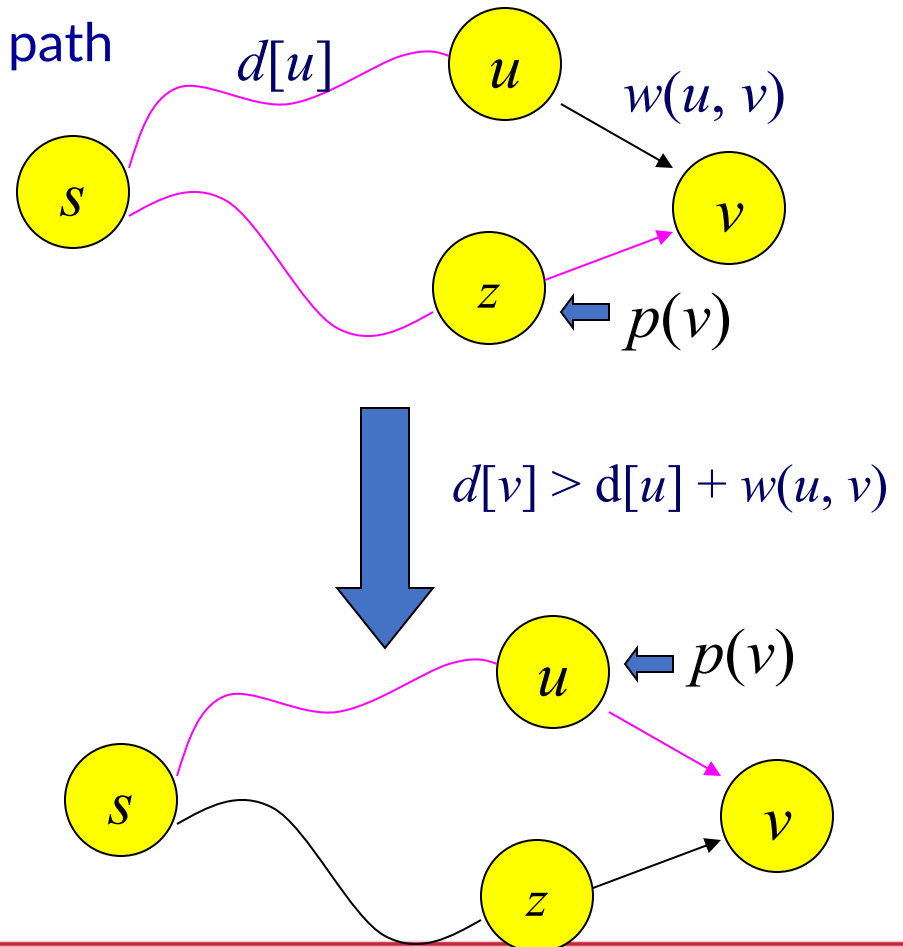
Building the shortest path from s to v :

Assume the current known shortest path from s to v : $s \dots z, v$

Relaxing an edge (u, v) means testing whether we can improve the shortest path to v found so far by going through u

$\text{Relax}(u, v)$

```
if ( $d[v] > d[u] + w(u, v)$ ) {  
     $d[v] \leftarrow d[u] + w(u, v)$   
     $p[v] \leftarrow u$   
}
```



Properties of Relaxation

Shortest path algorithms differ in

- *how many times* they relax each edge, and
- *the order* in which they relax edges

```
Relax( $u, v$ )  
  if ( $d[v] > d[u] + w(u, v)$ )  
  {  
     $d[v] \leftarrow d[u] + w(u, v)$   
     $p[v] \leftarrow u$   
  }
```

Single source shortest path

1. **Bellman-Ford algorithm**
2. Dijkstra algorithm

Bellman-Ford algorithm



Richard Bellman
1920-1984



Lester R. Ford, Jr.
1927-2017

Bellman-Ford algorithm

Bellman-Ford algorithm is used to find the shortest path from a vertex s to each other vertex in the graph.

- **Input:** A directed graph $G=(V,E)$ and weight matrix $w[u,v] \in R$ where $u,v \in V$, source vertex $s \in V$;
 - G does not contain **negative-weight cycle**
- **Output:** Each $v \in V$
 - $d[v] = \delta(s, v)$; Length of the shortest path from s to v
 - $p[v]$ - the predecessor of v in this shortest path from s to v .

Bellman-Ford algorithm: Full version

Bellman-Ford(*G*, *w*, *s*)

// Step 1: Initialize shortest paths of with at most 0 edges

1. **Initialize-Single-Source**(*G*, *s*)

/* Step 2: Calculate shortest paths with at most *i* edges from shortest paths with at most *i*-1 edges */

2. **for** *i* in range (1, |*V*|)

3. **for** each edge (*u*, *v*) ∈ *E*

4. Relax(*u*, *v*)

5. **for** each edge (*u*, *v*) ∈ *E*

6. **if** *d*[*v*] > *d*[*u*] + *w*(*u*, *v*)

7. **return** False // there is a negative cycle

8. **return** True

Relax(*u*, *v*)

```
if d[v] > d[u] + w(u, v) {  
    d[v] = d[u] + w[u, v] ;  
    p[v] = u ;  
}
```

Initialize-Single-Source(*G*, *s*)

```
for v ∈ V \ s {  
    d[v] = ∞;  
    p[v] = Null;  
}  
p[s] = Null; d[s] = 0;
```



Bellman-Ford algorithm: Full version

Bellman-Ford(G, w, s)

```
// Step 1: Initialize shortest paths of  $w$  with at most 0 edges
```

1. Initialize-Single-Source(G, s) $\longrightarrow O(|V|)$

```
/* Step 2: Calculate shortest paths with at most i edges from shortest
   paths with at most i-1 edges */
```

2. **for** i in range (1, |V|) $\longrightarrow O(|V||E|)$

```

3.   for each edge (u, v) ∈ E
4.       Relax(u, v)

```

Lines (2-4): First nested **for-loop** performs $|V|-1$ relaxation iterations; relax every edge at each iteration ☐ Running time $O(|V||E|)$

5. **for** each edge $(u, v) \in E$ $\longrightarrow O(|E|)$

6. **if** $d[v] > d[u] + w(u, v)$

```
7.         return False // there is a negative cycle
```

```
8.     return True
```

- Running time $O(|V||E|)$
- Memory space: $O(|V|^2)$



Bellman-Ford algorithm

Bellman-Ford(G, w, s)

```
1. Initialize-Single-Source( $G, s$ )  $\longrightarrow O(|V|)$ 
2. for  $i$  in range  $(1, |V|)$   $\longrightarrow O(|V||E|)$ 
3.     for each edge  $(u, v) \in E$ 
4.         Relax( $u, v$ )
5. for each edge  $(u, v) \in E$   $\longrightarrow O(|E|)$ 
6.     if  $d[v] > d[u] + w(u, v)$ 
7.         return False // there is a negative cycle
8. return True
```

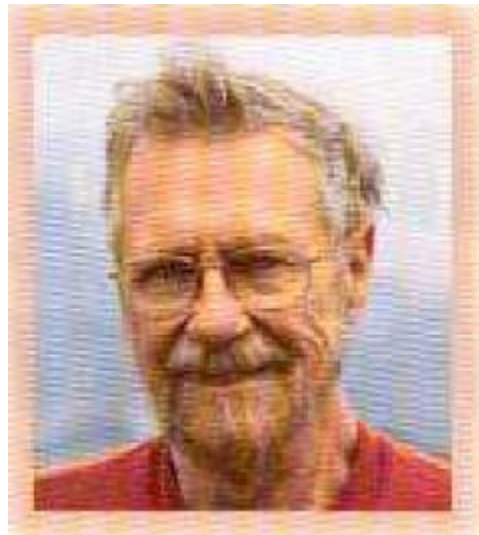
if Ford-Bellman has not converged after $|V| - 1$ iterations, then there cannot be a shortest path tree, so there must be a negative weight cycle.

Single source shortest path

1. Bellman-Ford algorithm
- 2. Dijkstra algorithm**

Dijkstra algorithm

- In case the weights on the edges are non-negative, the algorithm proposed by Dijkstra is more efficient than the Ford-Bellman algorithm.
- Algorithms are built by labeling vertices. The label of the vertices is initially temporary. At each iteration there is a temporary label that becomes a permanent label. If the label of a vertex u becomes fixed, $d[u]$ gives us the length of the shortest path from the source s to u . Algorithm ends when the labels of all vertices become fixed.



Edsger W.Dijkstra
(1930-2002)

Dijkstra algorithm

- **Input:** A directed graph $G=(V,E)$ and weight matrix $w[u,v] \geq 0$ where $u,v \in V$, source vertex $s \in V$;
 - G does not have **negative-weight cycle**
- **Output:** Each $v \in V$
 - $d[v] = \delta(s, v)$; Length of the shortest path from s to v
 - $p[v]$ - the predecessor of v in this shortest path from s to v .

Use greedy algorithm:

Maintain a set S of vertices for which we know the shortest path

At each iteration:

- grow S by one vertex , choosing shortest path through S to any other vertex not in S
- If the cost from S to any other vertex has decreased, update it

Dijkstra algorithm

```
Dijkstra ( ) {  
    for v ∈ V { // Initialize  
        d[v] = w[s,v] ;  
        p[v] = s;  
    }  
    d[s] = 0; S = {s}; // S: the set of vertices with fixed label  
    (shortest path from s to it has been found)  
    T = V \ {s}; // T: the set of vertices with temporary label  
    while (T ≠ ∅){ //Loop  
        Find vertex u ∈ T satisfying d[u] = min{ d[z] : z ∈ T};  
        T = T \ {u}; S = S ∪ {u}; //Fixed label of vertex u  
        for v ∈ adj[u] and v ∈ T do  
            if (d[v] > d[u] + w[u,v]){  
                d[v] = d[u] + w[u,v] ;  
                p[v] = u ;  
            }  
        }  
    }
```

Dijkstra algorithm

```
void Dijkstra ( )
```

```
{
```

```
    for    v  $\in$  V do { // Initialize
```

```
        d[v] = w[s,v] ;
```

```
        p[v]=s;
```

```
    }
```

```
    d[s] = 0;    S = {s};
```

```
    T = V \ {s};
```

```
    while  (T  $\neq$   $\emptyset$ )  {
```

```
        //Loop
```

```
        Find vertex  u  $\in$  T satisfying  d[u] = min{ d[z] :  z  $\in$  T};
```

```
        T =  T \ {u};  S= S  $\cup$  {u};
```

```
        for    v  $\in$  adj[u] and v  $\in$  T
```

```
            if    (d[v] > d[u] + w[u,v])
```

```
            {
```

```
                d[v] = d[u] + w[u,v] ;
```

```
                p[v] = u ;
```

```
            }
```

```
    }
```

```
}
```

- $O(|V|^2)$ operations

- $(|V|-1)$ iterations: 1 for each vertex u added to the distinguished set S.

- $(|V|-1)$ iterations: for each adjacent vertex of the one added to the distinguished set.

Shortest path problems

1. Bellman-Ford algorithm

Edge weight: $>, <, = 0$

Running time: $O(|V||E|)$

Shortest path in the graph having cycle

Non-negative length cycle

2. Dijkstra algorithm

Edge weight ≥ 0

Running time: $O(|V|^2)$

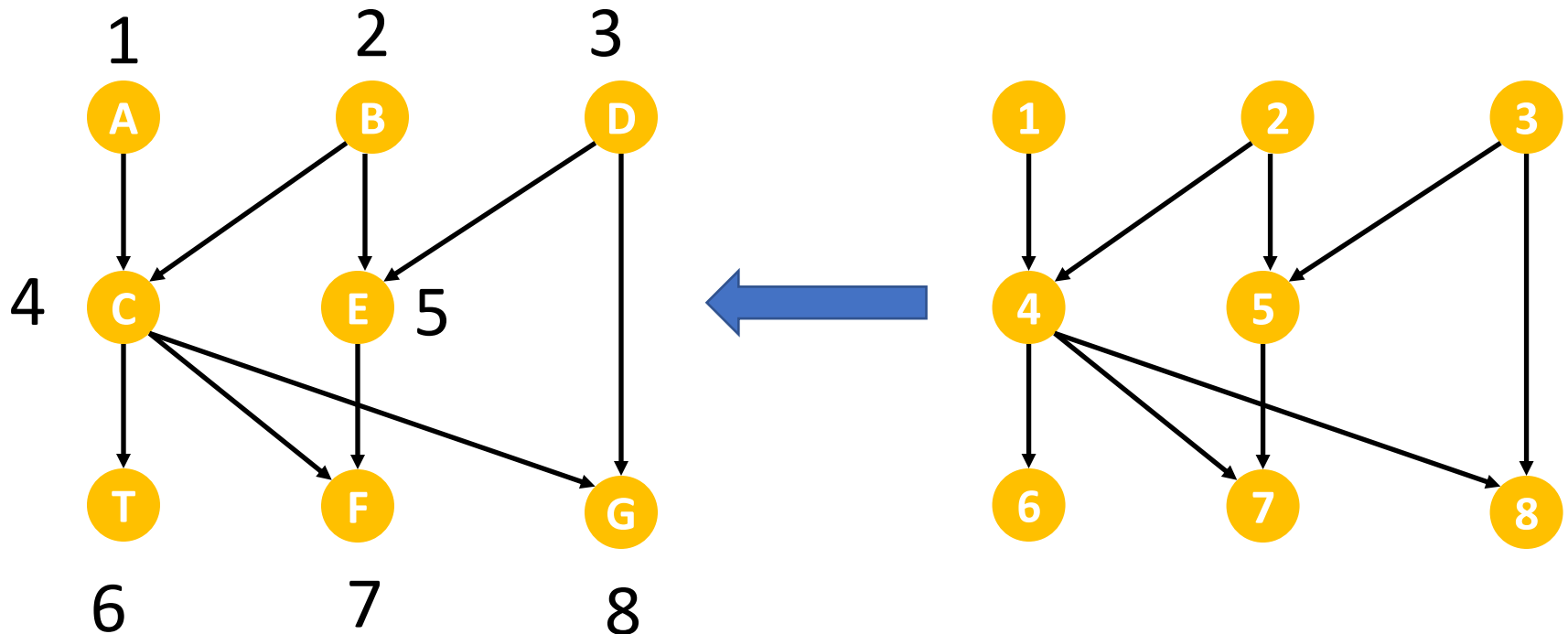
3. Shortest path in the directed graph with no cycles
(Directed acyclic graph (DAG))

Edge weight: $>, <, = 0$

Running time: $O(|E|)$

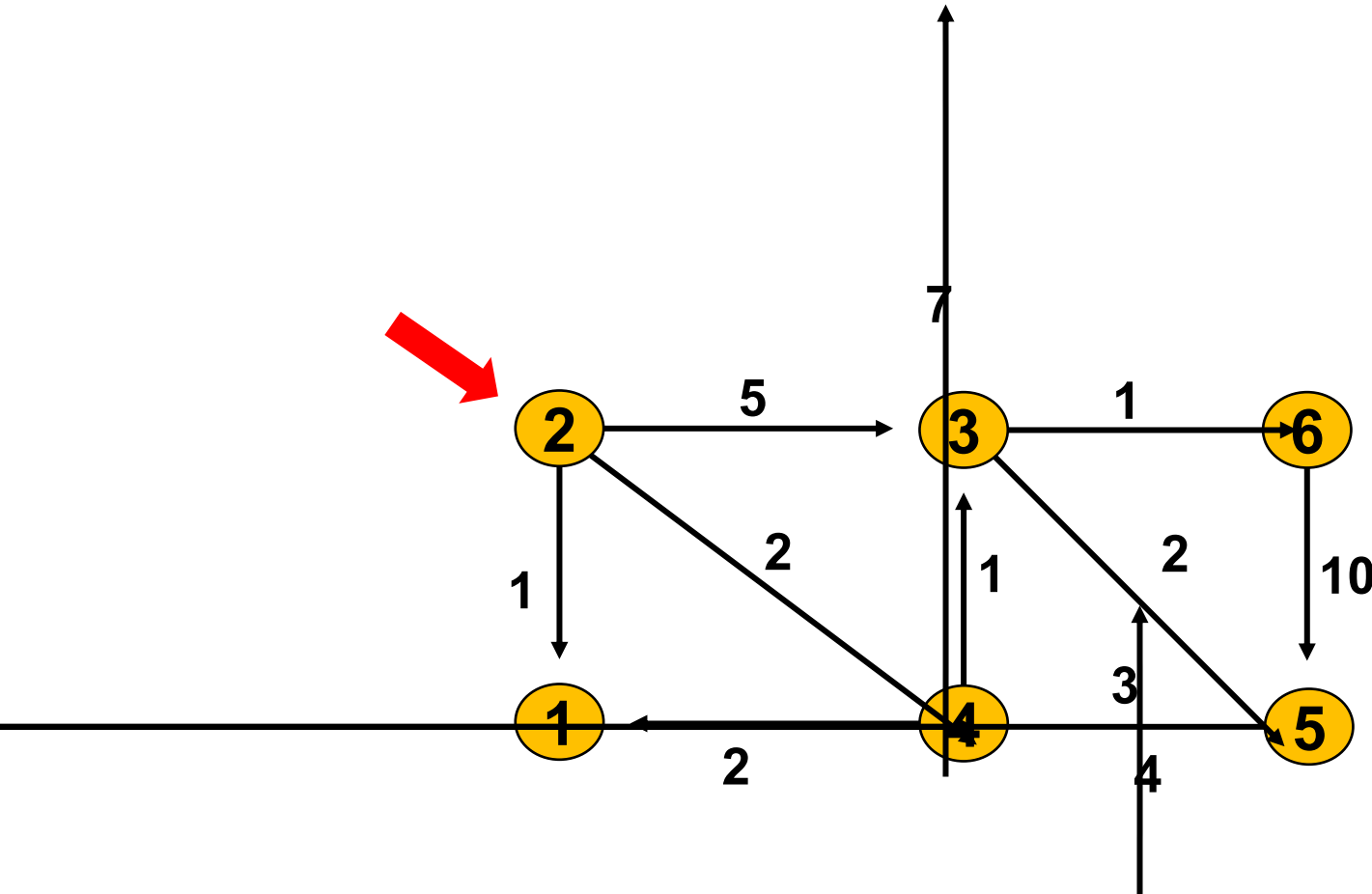
Single-Source Shortest Paths in DAGs

A **topological sort** or **topological ordering** of a DAG is a [linear ordering](#) of its vertices such that for every directed edge (u, v) from vertex u to vertex v , u comes before v in the ordering. (In other words: its vertexes can be numbered so that each directed edge starting from the vertex with the smaller index to the vertex with a larger index)



Topological sorting algorithm

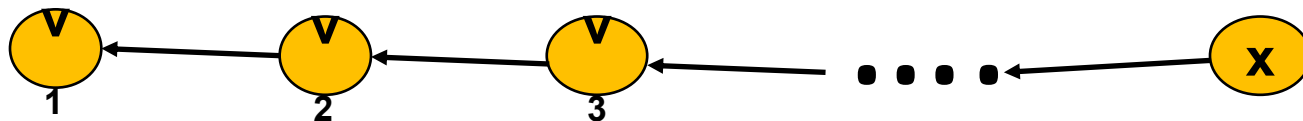
- We see that: *In the DAG, there always exists a vertex with in-degree = 0*



Topological sorting algorithm

- We see that: *In the DAG, there always exists a vertex with in-dgree = 0*

Indeed, starting at vertex v_1 if there is an incoming edge to it from vertex v_2 then we move to v_2 . If there is an edge from v_3 to v_2 , then we switch to v_3 , ... Since there is not any cycle in the graph, so after a finite number of such transfers we have to go to the vertex without incoming edge.



- **Topological sorting algorithm:**

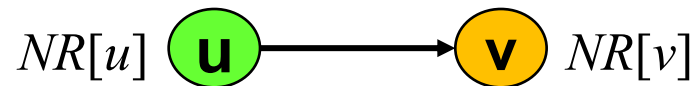
First, finding all vertices with in-dgree = 0. We index these vertices starting from 1.

Next, removing from graphs vertices that have just been indexed together with the edge going out of them, we get a new graph also without cycle, and we again starting index vertices on this new graph.

The process is repeated until all vertices of the graph has been indexed.

Topological sorting algorithm

- **Input:** DAG $G=(V,E)$ with the adjacent list $Adj(v)$, $v \in V$.
- **Output:** For each $v \in V$ the index $NR[v]$ satisfying: Each directed edge (u, v) : $NR[u] < NR[v]$.



Single-Source Shortest Paths in DAGs

Shortest paths are always *well-defined* in *DAGS*

- no cycles => no negative-weight cycles even if there are negative-weight edges

In a DAG:

- Every path is a subsequence of the topologically sorted vertex order
- If we do topological sort and process vertices in that order
- We will process each path in forward order : Never relax edges out of a vertex until have processed all edges into the vertex. Thus, just 1 iteration is sufficient

```
DAG-SHORTEST PATHS(G, s)
  L = TOPOLOGICALLY-SORT(G);
  INIT(G, s)
  for each vertex u taken in L do
    for each v ∈ Adj[u] do
      RELAX(u, v)
```

- Topological sorting: $O(|E|)$
- Initialized-Single-Source: $O(|E|)$
- Nested for-loop: each edge is “traversed” exactly once. Hence, it takes $O(|E|)$ time.

Hence, total running time: $O(|E|)$

Single-Source Shortest Paths in DAGs

- **Input:** DAG $G = (V, E)$ with topological sorting $V = \{v[1], v[2], \dots, v[n]\}$.
Each directed edge $(v[i], v[j]) \in E$, we have $i < j$.
The adjacent list $\text{Adj}(v)$, $v \in V$.
- **Output:** The shortest path from $v[1]$ to all other vertices stored in the array $d[v[i]]$, $i = 2, 3, \dots, n$

```
DAG-SHORTEST PATHS( $G$ ,  $v[1]$ )  
  INIT( $G$ ,  $v[1]$ )  
  for  $i = 2$  to  $n$  do  
    for each  $x \in \text{Adj}[v[i]]$  do  
      RELAX( $v[i]$ ,  $x$ )
```

Single-Source Shortest Paths in DAGs

Critical_Path ()

{

$d[v[1]] = 0;$

for j in range $(2, n+1)$ $d[v[j]] = \infty;$

for $v[j] \in \text{Adj}[v[1]]$

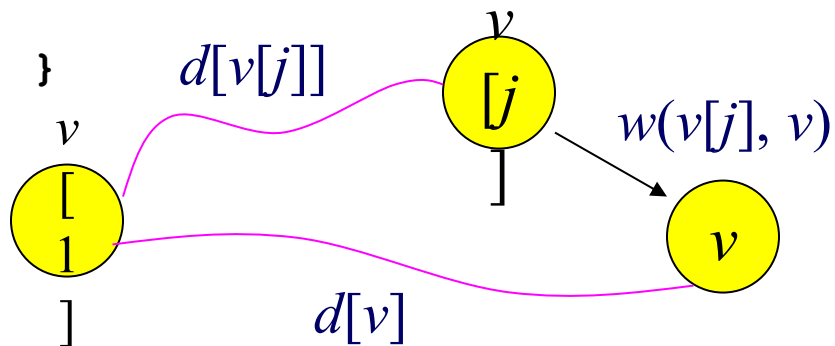
$d[v[j]] := w(v[1], v[j]) ;$

for j in range $(2, n+1)$

for $v \in \text{Adj}[v[j]]$

$\text{Relax}(v[j], v)$

}



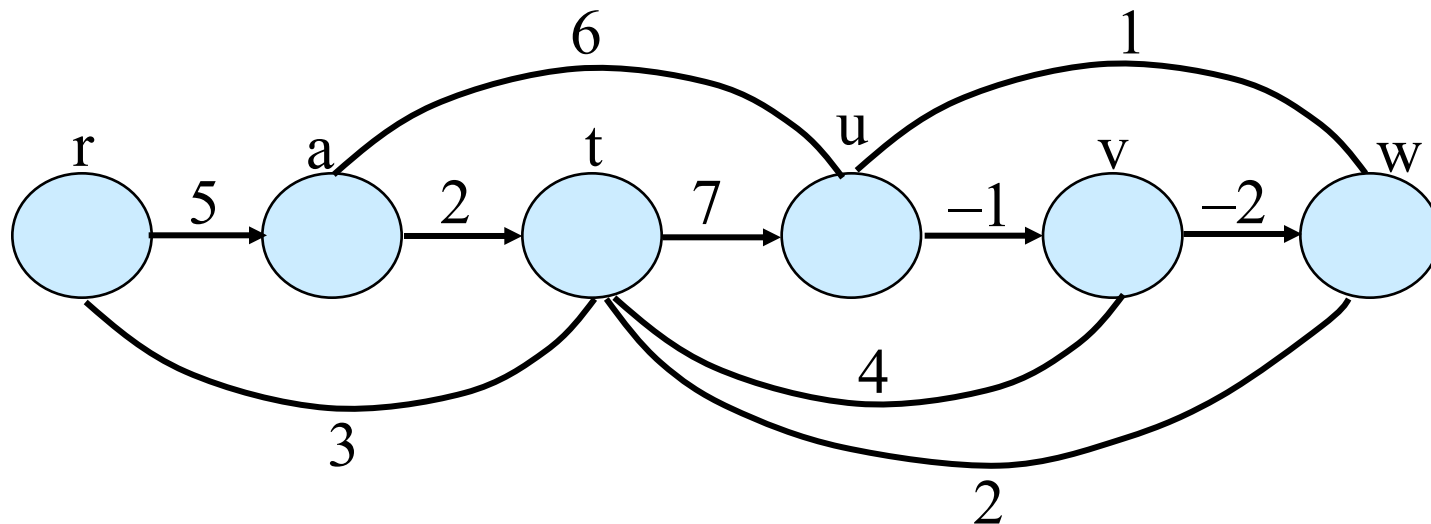
$d[v] = \min(d[v], d[v[j]] + w(v[j], v));$

Initialization:
Init(G, v[1])

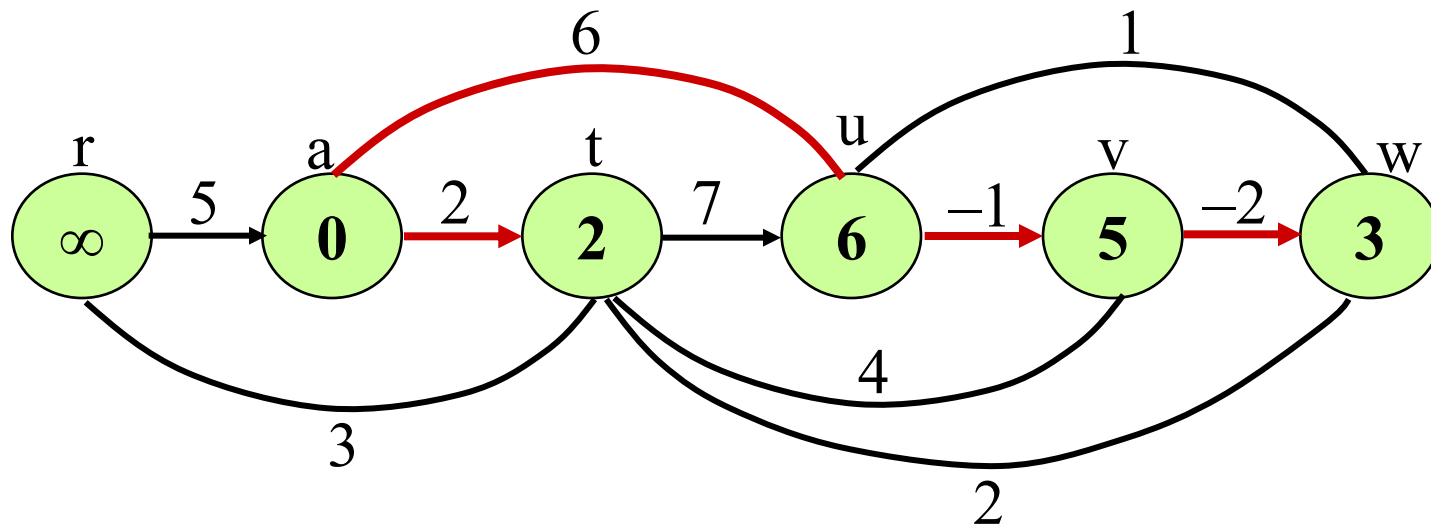
Running time: $O(|E|)$, since each edge is traversed exactly once

Example

Find the shortest path from **a** to each other vertices of the DAG graph where vertices are already topological order



Example



Result: The shortest path tree from a represented by the red edges

Shortest-Path Variants

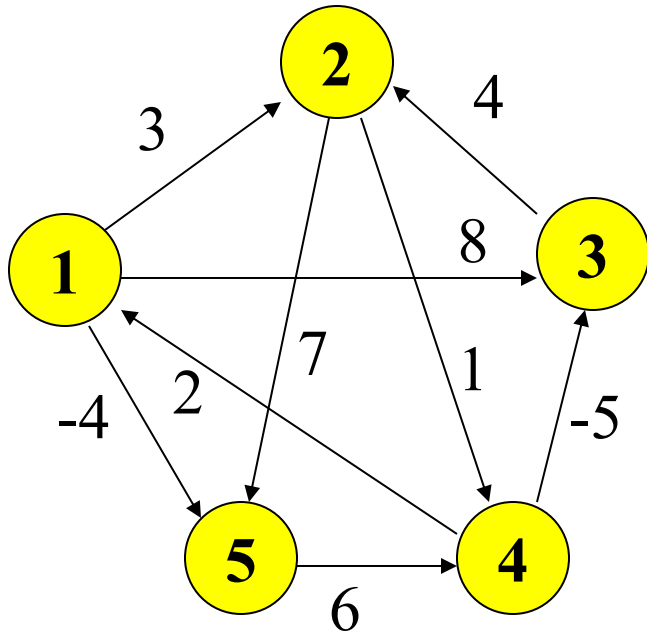
- **Single-source shortest-paths problem**
 - Find a shortest path from a given source (vertex s) to each of the vertices.
- **Single-destination shortest-paths problem**
 - Find a shortest path to a given *destination* vertex t from each vertex v .
- **Single-pair shortest-path problem**
 - Given two vertices, find a shortest path between them. Solution to single-source problem solves this problem efficiently, too.
- **All-pairs shortest-paths problem**
 - Find a shortest path from u to v for every pair of vertices u and v

All pairs shortest-paths

Problem Given directed graph $G = (V, E)$, with weight on each edge e is $w(e)$,
for each pair of vertices u, v of V , find the shortest path from u to v .

- ✦ Input: *weight matrix*
- ✦ Output *matrix*: element at row u column v is the length of the shortest path from u to v .
- ✦ Allow negative-weight edge
- ✦ **Assumption: None negative-length cycle**

Example



Input

Weight matrix $W_{n \times n} = (w)_{ij}$ where

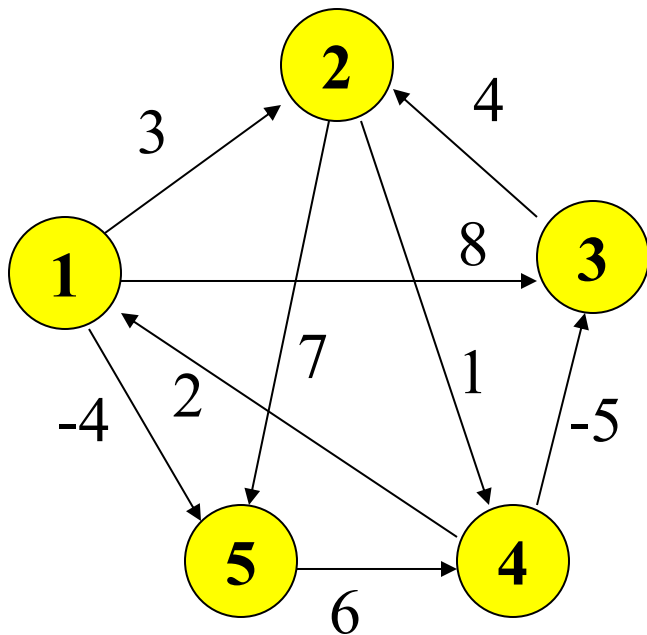
$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ w(i, j) & \text{if } i \neq j \text{ \& } (i, j) \in E \\ \infty & \text{otherwise} \end{cases}$$

$$W_{5 \times 5} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

Example

Output

Matrix: element at row u column v is the length of the shortest path from u to v .



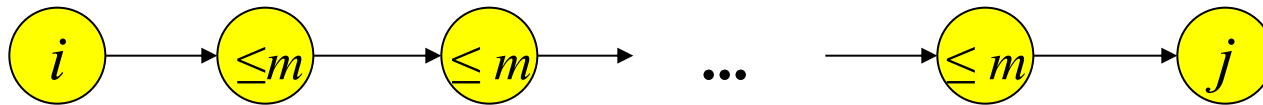
Shortest path from **1** to **2** : 1 - 5 - 4 - 3 - 2

$$\begin{matrix} & \downarrow & \nearrow & & \\ & \mathbf{1} & & = -4 + 6 - 5 + 4 & \\ \left(\begin{array}{ccccc} 0 & \mathbf{1} & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ \mathbf{8} & 5 & 1 & 6 & 0 \end{array} \right) & & \nearrow & & \end{matrix}$$

Shortest path from **5** to **1** : 5 - 4 - 1

Floyd-Warshall algorithm

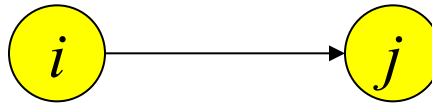
$d_{ij}^{(m)}$ = length of the shortest path from i to j using intermediate vertices in the vertex set $\{1, 2, \dots, m\}$.



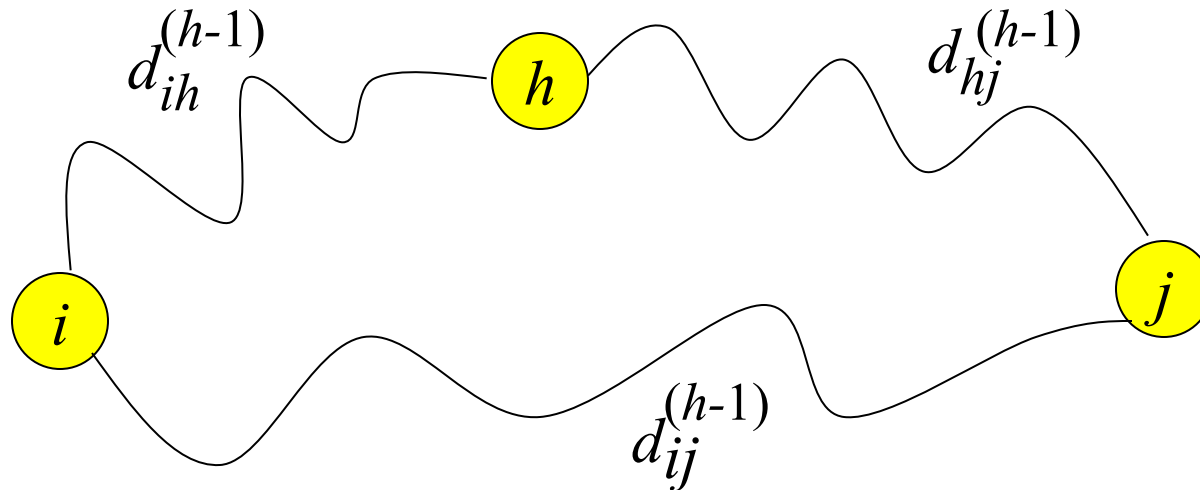
Graph with n vertices $\{1, 2, \dots, n\}$ □ length of the shortest path from i to j $d_{ij}^{(n)}$ is

Recursive formula computed $d(h)$

★ $d_{ij}^{(0)} = w_{ij}$



★ $d_{ij}^{(h)} = \min (d_{ij}^{(h-1)}, d_{ih}^{(h-1)} + d_{hj}^{(h-1)})$ if $h \geq 1$



Floyd-Warshall algorithm

```
void Floyd-Warshall( $n, W$ )
```

```
{
```

```
     $D^{(0)} \leftarrow W$ 
```

```
    for ( $k = 1; k \leq n; k++$ )
```

Path going through only intermediate vertices selected from $\{1, 2, \dots, k\}$

```
        for ( $i = 1; i \leq n; i++$ )
```

```
            for ( $j = 1; j \leq n; j++$ )
```

All pairs (i, j)

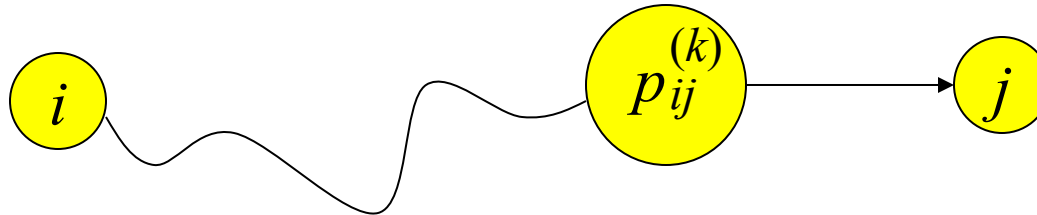
```
    return  $D^{(n)}$ ;  $d_{ij}^{(k)} \leftarrow \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
```

```
}
```

Running time $\Theta(n^3)$!

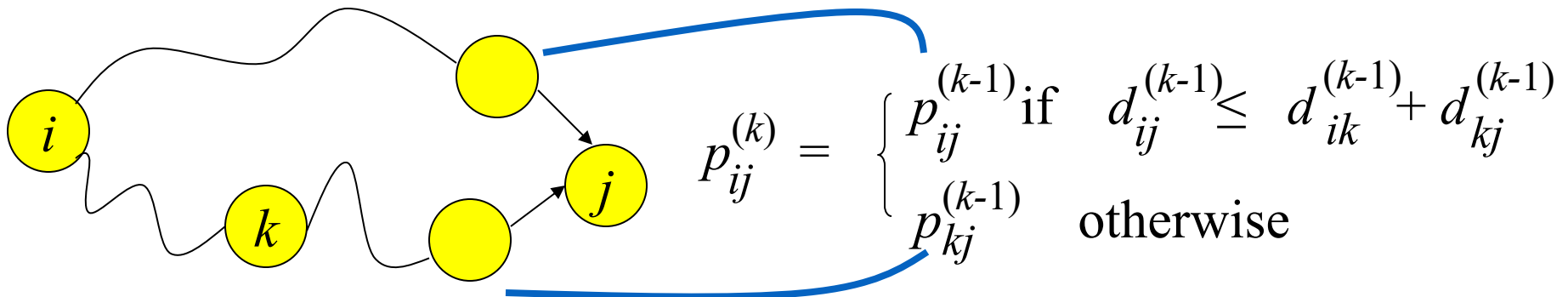
Build the shortest path

Predecessor matrix $P^{(k)} = (p_{ij}^{(k)})$:

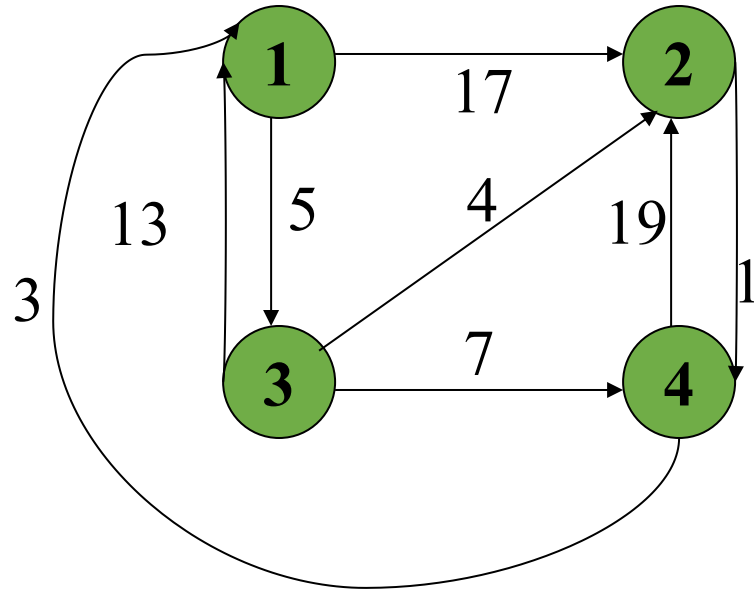


Shortest path from i to j going through intermediate vertices only selected from $\{1, 2, \dots, k\}$.

$$p_{ij}^{(0)} = \begin{cases} i, & \text{if } (i, j) \in E \\ \text{Nil}, & \text{if } (i, j) \notin E \end{cases}$$



Example: Find shortest path between every pairs of vertices



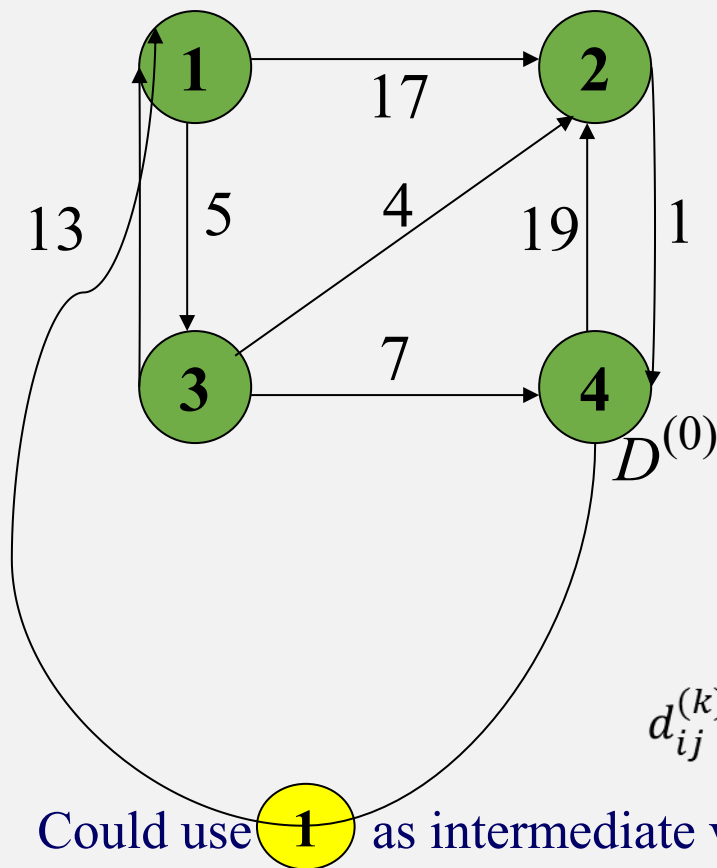
Input

Weight matrix $W_{n \times n} = (w)_{ij}$ where

$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ w(i, j) & \text{if } i \neq j \text{ \& } (i, j) \in E \\ \infty & \text{otherwise} \end{cases}$$

$$W_{4 \times 4} = \begin{pmatrix} 0 & 17 & 5 & \infty \\ \infty & 0 & \infty & 1 \\ 13 & 4 & 0 & 7 \\ 3 & 19 & \infty & 0 \end{pmatrix}$$

3



$$D^{(0)} \leftarrow W$$

$$p_{ij}^{(0)} = \begin{cases} i, & \text{nếu } (i, j) \in E \\ \text{Nil}, & \text{nếu } (i, j) \notin E \end{cases}$$

$$D^{(0)} \begin{pmatrix} 0 & 17 & 5 & \infty \\ \infty & 0 & \infty & 1 \\ 13 & 4 & 0 & 7 \\ 3 & 19 & \infty & 0 \end{pmatrix}$$

$$P^{(0)} \begin{pmatrix} \text{Nil} & 1 & 1 & \text{Nil} \\ \text{Nil} & \text{Nil} & \text{Nil} & 2 \\ 3 & 3 & \text{Nil} & 3 \\ 4 & 4 & \text{Nil} & \text{Nil} \end{pmatrix}$$

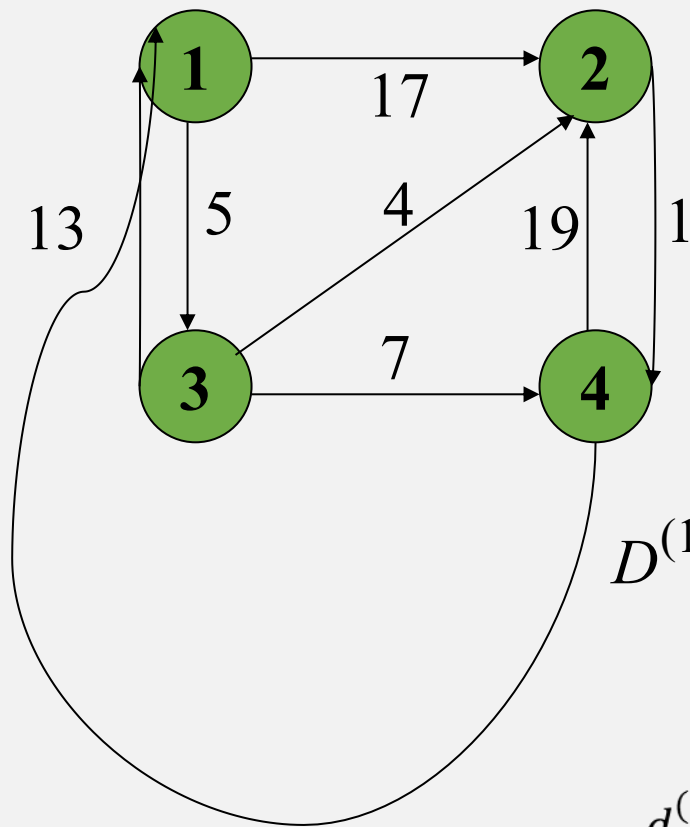
$$d_{ij}^{(k)} \leftarrow \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$$

Could use **1** as intermediate vertex:

$$D^{(1)} \begin{pmatrix} 0 & 17 & 5 & \infty \\ \infty & 0 & \infty & 1 \\ 13 & 4 & 0 & 7 \\ 3 & 19 & 8 & 0 \end{pmatrix}$$

$$P^{(1)} \begin{pmatrix} \text{Nil} & 1 & 1 & \text{Nil} \\ \text{Nil} & \text{Nil} & \text{Nil} & 2 \\ 3 & 3 & \text{Nil} & 3 \\ 4 & 4 & 1 & \text{Nil} \end{pmatrix}$$

3



$$D^{(0)} \leftarrow W$$

$$p_{ij}^{(0)} = \begin{cases} i, & \text{nếu } (i,j) \in E \\ \text{Nil}, & \text{nếu } (i,j) \notin E \end{cases}$$

$$D^{(1)} = \begin{pmatrix} 0 & 17 & 5 & \infty \\ \infty & 0 & \infty & 1 \\ 13 & 4 & 0 & 7 \\ 3 & 19 & \mathbf{8} & 0 \end{pmatrix}$$

$$P^{(1)} = \begin{pmatrix} \text{Nil} & 1 & 1 & \text{Nil} \\ \text{Nil} & \text{Nil} & \text{Nil} & \mathbf{2} \\ 3 & 3 & \text{Nil} & 3 \\ 4 & 4 & \mathbf{1} & \text{Nil} \end{pmatrix}$$

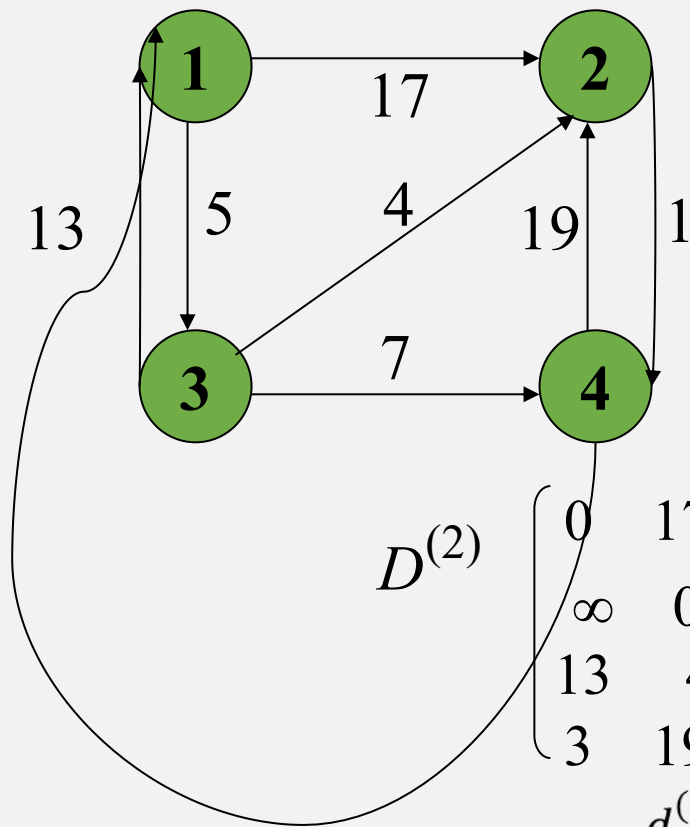
$$d_{ij}^{(k)} \leftarrow \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$$

Could use **2** as intermediate vertex:

$$D^{(2)} = \begin{pmatrix} 0 & 17 & 5 & \mathbf{18} \\ \infty & 0 & \infty & 1 \\ 13 & 4 & 0 & \mathbf{5} \\ 3 & 19 & 8 & 0 \end{pmatrix}$$

$$P^{(2)} = \begin{pmatrix} \text{Nil} & 1 & 1 & 2 \\ \text{Nil} & \text{Nil} & \text{Nil} & 2 \\ 3 & 3 & \text{Nil} & \mathbf{2} \\ 4 & 4 & \mathbf{1} & \text{Nil} \end{pmatrix}$$

3



$$D^{(0)} \leftarrow W$$

$$p_{ij}^{(0)} = \begin{cases} i, & \text{nếu } (i,j) \in E \\ \text{Nil}, & \text{nếu } (i,j) \notin E \end{cases}$$

$$D^{(2)} = \begin{pmatrix} 0 & 17 & 5 & 18 \\ \infty & 0 & \infty & 1 \\ 13 & 4 & 0 & 5 \\ 3 & 19 & 8 & 0 \end{pmatrix}$$

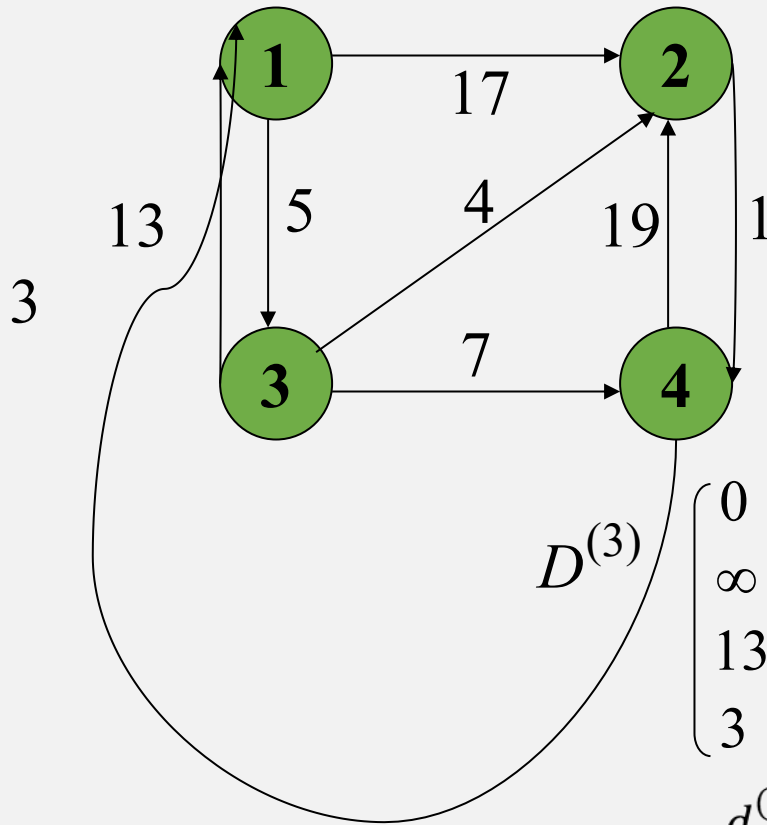
$$P^{(2)} = \begin{pmatrix} \text{Nil} & 1 & 1 & \text{Nil} \\ \text{Nil} & \text{Nil} & \text{Nil} & 2 \\ 3 & 3 & \text{Nil} & 2 \\ 4 & 4 & 1 & \text{Nil} \end{pmatrix}$$

$$d_{ij}^{(k)} \leftarrow \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$$

Could use 3 as intermediate vertex:

$$D^{(3)} = \begin{pmatrix} 0 & \mathbf{9} & 5 & \mathbf{10} \\ \infty & 0 & \infty & 1 \\ 13 & 4 & 0 & 5 \\ 3 & \mathbf{12} & 8 & 0 \end{pmatrix}$$

$$P^{(3)} = \begin{pmatrix} \text{Nil} & \mathbf{3} & 1 & \mathbf{2} \\ \text{Nil} & \text{Nil} & \text{Nil} & 2 \\ 3 & 3 & \text{Nil} & 2 \\ 4 & \mathbf{3} & 1 & \text{Nil} \end{pmatrix}$$



$$D^{(0)} \leftarrow W$$

$$p_{ij}^{(0)} = \begin{cases} i, & \text{nếu } (i,j) \in E \\ \text{Nil}, & \text{nếu } (i,j) \notin E \end{cases}$$

$$D^{(3)} = \begin{pmatrix} 0 & 9 & 5 & 10 \\ \infty & 0 & \infty & 1 \\ 13 & 4 & 0 & 5 \\ 3 & 12 & 8 & 0 \end{pmatrix} \quad P^{(3)} = \begin{pmatrix} \text{Nil} & 3 & 1 & 2 \\ \text{Nil} & \text{Nil} & \text{Nil} & 2 \\ 3 & 3 & \text{Nil} & 2 \\ \textcolor{red}{4} & 3 & \textcolor{blue}{1} & \text{Nil} \end{pmatrix}$$

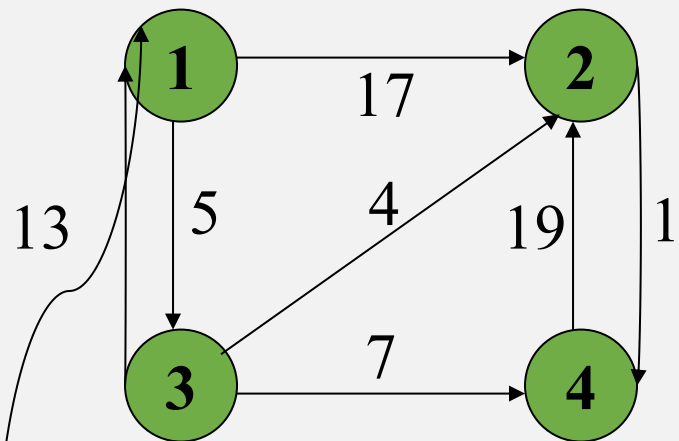
$$d_{ij}^{(k)} \leftarrow \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$$

Could use 4 as intermediate vertex:

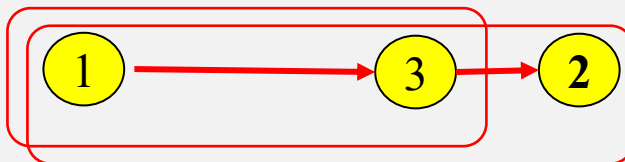
$$D^{(4)} = \begin{pmatrix} 0 & 9 & 5 & 10 \\ \textcolor{red}{4} & 0 & \textcolor{red}{9} & 1 \\ \textcolor{red}{8} & 4 & 0 & 5 \\ 3 & 12 & 8 & 0 \end{pmatrix}$$

$$P^{(4)} = \begin{pmatrix} \text{Nil} & 3 & 1 & 2 \\ \textcolor{red}{4} & \text{Nil} & \textcolor{blue}{1} & 2 \\ \textcolor{red}{4} & 3 & \text{Nil} & 2 \\ 4 & 3 & 1 & \text{Nil} \end{pmatrix}$$

3



Shortest path from 1 to 2:
Length = 9



$$D^{(4)} = \begin{pmatrix} 0 & 9 & 5 & 10 \\ 4 & 0 & 9 & 1 \\ 8 & 4 & 0 & 5 \\ 3 & 12 & 8 & 0 \end{pmatrix} \quad P^{(4)} = \begin{pmatrix} \text{Nil} & 3 & 1 & 2 \\ 4 & \text{Nil} & 1 & 2 \\ 4 & 3 & \text{Nil} & 2 \\ 4 & 3 & 1 & \text{Nil} \end{pmatrix}$$

A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in a way that creates a sense of depth and movement, resembling a spiral or a series of concentric circles that are slightly offset from each other.

HUST

THANK YOU !